

X5 项目技术难点与解决方案

项目范围: insv_to_project 11 步 MATLAB pipeline (后端) + VideoGPXAnnotationTool_v30 GUI。

28 条条目

目录

1. .insv 私有 trailer 流抽取 GPS/IMU
2. exiftool 13.59 RequestAll=3 触发 perl bug
3. exiftool 间歇崩与 Defender 排除
4. QuickTime:CreateDate 无 Z 后缀但值是 UTC
5. .insv 分辨率检测: dual-fisheye → equirect 推算
6. X5 1Hz timelapse 不变量
7. video_start_epoch 必现算, 不能复用旧值
8. IMU/GPS 时间对齐: mean_of_ends ±200s
9. GPSTime 混 .X / 无小数 + duplicate ts
10. exiftool IMU 输出 1Hz, 200Hz 需 binary parser
11. MediaSDK 公开 API 无 GPS, 只能 exiftool
12. stitch_to_frames 崩 → MediaSDKTest.exe
13. insv_split.ps1 GPSTime 正则锚定 Z
14. GPS 时间用秒 since epoch 排序
15. PowerShell 5.1 不支持 PS6+ `?.`
16. MATLAB fopen 写 binary 0xFF 0xD8 被吃
17. MATLAB fread 'uint64' 只读 4 bytes
18. .NET BinaryReader.ReadBytes 大文件 OOM
19. uigetfile MultiSelect 字符串只接受 on/off
20. GUI 集成必改 prompt_idempotent.m 静默 questdlg
21. V30 uifigure 标题栏高度让位问题
22. V30 WindowState='maximized' 模式敲定
23. 离线地图坐标与 GPS 坐标系不对应, 需转换坐标系
24. Mapsui 五个实测坑 (WPF/Polygon/版本/TMS)
25. V30.24+ 八连 bug: 白底/Y倒/范围过小
26. 手写投影公式必踩纬度方向偏差
27. 30+ 诊断脚本的五轮迭代历程
28. 自带 SDK 不支持多线程, 自实现 4 线程 wrapper (1~16 线程对比)

01 Insta360 X5 .insv 私有 trailer 流抽取 GPS/IMU

exiftool trailer GPS

难点 Insta360 X5 的 GPS/加速度计/陀螺仪数据存在文件尾部一个 ~30–40MB 的私有 `trailer` 流里，**不在标准 moov/inst MP4 树中**。exiftool 默认会把这条流跳掉 (warn: "Use ExtractEmbedded option to extract timed metadata from Insta360 trailer")，结果只看到两个 AVC/HVC 视频轨，没有 GPS。

方案 关键参数 `-ee` (ExtractEmbedded) 必须加。大文件 (≥100MB 或 ≥1min) 走 `-ee -G1 -a -s -m`；小文件 (类似 005, 38MB) 仍可加 `-api RequestAll=3`。

```
ExifTool.exe -api largefilesupport -ee -G1 -a -s -m FILE.insv
```

关键点 无 `-ee` 看不到 GPS；无 `-m` 走不到 per-frame 路径 (被 20k sample cap 警告当成 fatal error 停掉)。

实测：VID_006 加 `-ee -G1 -a -s -m` 跑出 169MB / 1.25M 行，4,003 GPS + 401,104 IMU + 413,094 TimeCode + 11,991 ExposureTime。

02 exiftool 13.59 -api RequestAll=3 触发 perl memory bug

exiftool bug per-frame

难点 exiftool 13.59 加 `-api RequestAll=3` 跑大 .insv (373MB 006) 时 perl 内部 state 被搞坏，死在不同 line (Get32u, DUPL_TAG, QuickTimeStream.pl) 都报 "Not an ARRAY/HASH reference" 或 "free unreferenced scalar"。16:07 偶然跑通一次 (出 169MB, 401k IMU)，17:30+ 必崩。

方案 两个 flag 的角色容易混：

- `-m` (`--ignoreMinorErrors`) = **per-frame 抽取的开关**。没 `-m` 走不到 per-frame 路径。
- `-api RequestAll=3` = **bug 触发器**。RequestAll=2 不会。

修法：去掉 `-api RequestAll=3`，保留 `-m`：

```
ExifTool.exe -api largefilesupport -ee -G1 -a -s -m FILE.insv <-- OK (5s 稳定 169MB)
ExifTool.exe -api largefilesupport -api RequestAll=3 -ee -G1 -a -s -m FILE.insv <-- NOT OK
```

关键点 看 Accel count 判断走的哪条路径：20,001 = 1Hz 标准；401k = per-frame。

实测：推荐命令 5s 稳定出 169MB / 1.25M 行 (与 16:07 lucky pass 等效)；RequestAll=3 路径 17:30+ 25% 概率崩。

03 exiftool 间歇崩 (0xC0000005 / exit 255 / 226 bytes) → tmpfile + retry + Defender 排除

exiftool 稳定性 Defender

难点 即便去掉 RequestAll=3, exiftool 13.59 仍 ~25% 概率崩 (同族 perl memory bug)。症状包括 0xC0000005 (STATUS_ACCESS_VIOLATION)、exit 255、stderr 沉默只写 226 bytes。

方案 run_exiftool_raw.m 内置多层保护 (2026-06-30 升级版)：

- **tmpfile + retry + sanity**: 写临时文件, 3 次重试 + 1KB sanity check。
- **stderr 捕获**: 失败时把 perl 死在哪写到 <meta>.err。
- **pre-warm 1MB**: 用 .NET FileStream 读头把 OS cache 暖起来。
- **max_attempts 3 → 5** ($P(\text{recover in } \leq 5) \approx 76\%$)。
- **pause 1 → 3 秒** (让 OS cache 在 retry 间清掉)。

根因家族头号嫌疑: Windows Defender 实时扫 .meta / .tmp 写入过程中夺走句柄 → exiftool crash。**用户必做**:

```
Add-MpPreference -ExclusionPath 'C:\i361'
Add-MpPreference -ExclusionPath 'C:\matlabX5'
Add-MpPreference -ExclusionPath 'S:\项目\手持单兵式铁路沿线巡检设备'
```

关键点 P(per attempt) 实际 25% 而非 50%; 3 retry 不够, 需 5+。真正 fix 是 Defender 排除, 不是 perl 本身。

降级路径: 若 5 次 + Defender 都失败, 回退 exiftool 12.40 (只能拿 Insta360 Parameters, 无 per-sample); 或工程方案 PyPI telemetry-parser (56-byte record + 末尾 32-byte magic, 完全 bypass exiftool)。

04 QuickTime:CreateDate 无 Z 后缀但值是 UTC

exiftool UTC 时间戳

难点 ExifTool.exe -s -s -s -QuickTime:CreateDate FILE.insv 对 X5 输出形如 2026:06:17 05:06:29 (无 Z), 但 value 是 UTC 不是 local。证据: 同文件的 FileCreateDate: 2026:06:17 13:06:29+08:00, 跟 QuickTime CreateDate 差 8h, 正好 CST→UTC。

方案 不要用 FileCreateDate 减 8h 来"修正"。直接按 UTC parse:

```
dt = datetime(s, 'InputFormat', 'yyyy:MM:dd HH:mm:ss', 'TimeZone', 'UTC');
epoch = posixtime(dt);
```

错误写法 (带 'Z') 会 parse 失败, 因为 Z 不在 s 中。

关键点 exiftool 对 QuickTime 类标签默认不打 Z (atom 内部就是 UTC ISO 字符串, 无 timezone 概念), 跟 FileCreateDate / FileModifyDate (文件系统) 行为不同。任何 QuickTime 标签 (TrackCreateDate、MediaCreateDate) 都按 UTC parse。

验证: VID_006 实测 QuickTime:CreateDate = 2026:06:17 05:06:29 → posixtime = 1781672789 (UTC)。

05 .insv 分辨率检测: dual-fisheye → equirect 推算

MediaSDK 检测 stitch

难点 `exiftool -s -s -s -ImageWidth -ImageHeight VID_006.insv` 返回 `3840\n3840`。但 MediaSDK stitch 出来的是 `7680x3840` equirect 全景。直接信 `exiftool` 的 `ImageWidth` 会导致抽帧输出错误分辨率。

方案 `detect_insv_resolution.m` 的检测规则:

1. `exiftool probe -ImageWidth -ImageHeight`
2. 若 $W == H$ (方形, 单鱼眼) → equirect = $2*W \times H$
3. 否则按 W,H 直用
4. 失败 fallback `[7680 3840]`

根因: .insv 在 ISO BMFF 里是 dual-track (Track1 + Track2), 每 track 是 3840x3840 的单鱼眼图, MediaSDK 把两鱼眼 blend 成 2:1 equirect, W 翻倍, H 不变。exiftool 不知道 stitch pipeline, 只报 per-track raw 尺寸。

关键点 任何要按 .insv "原始分辨率" 抽帧的场景 (不指定 `-output_size` 或要 detect), 都用 `detect_insv_resolution.m`。

实测: VID_006 exiftool → 3840×3840, 推算 → 7680×3840 ✓; MediaSDKTest -output_size 7680x3840 → 400 jpgs, imfinfo 验 Width=7680 Height=3840 ✓。

06 X5 1Hz timelapse 不变量 (每视频帧 = 真实 1 秒)

时序 设备不变量

难点 Insta360 X5 在 device 上设 "1 photo/sec" 时间轴模式时，拍出来的 .insv 视频里**每 1 帧对应真实 1 秒**。容器头的 fps 字段通常还是 24/30，但实际帧间隔由 device 写入的真实数据决定。容易把 video container fps (24/30) 跟 device 拍帧率 (1Hz) 混了。

方案 整个时间轴 pipeline 都按这个不变量设计：

- `scripts/extract_jpgs.m:19` → `stitch_to_frames -frames -soft_decode` (无帧号) = 抽全帧 = 1 帧 1 秒
- `scripts/write_jpgs_exif.m:18` → `ts = video_start_epoch + (i - 1)`
- `scripts/match_gps_pps.m:11` → `ts = video_start_epoch + (0:M-1)'` (1Hz 对齐)
- `scripts/detect_highlights.m:15` → `(ts(i) - 10, ts(i))` past window (10s 窗口 = 过去 10 张 jpg = 真实 10 秒)

实际编码：device 1Hz 拍 N 张原始图，压成 30fps 视频 (不是 1fps)。视频长度 = N/30 秒，但 N 帧 = N 张原始图，1:1 映射。006 (.insv 13.33s @ 30fps) 实际拍了 6 分 40 秒 1Hz 拿 400 张图。

关键点 算 frame count 按 duration (N 秒 ≈ N 帧)，不按 fps × duration。006 应该出 ≈ 13 张 jpg (按 1Hz timelapse)，不是 400。

校验：`stitch_to_frames -frames 0` 抽第 1 帧，跟 `video_start_epoch` 对齐，误差应在 sub-second 内。

07 video_start_epoch 必现算，不能复用旧值

UTC 回归 验证

难点 VID_006.insv `video_start_epoch = 1781672789` (2026-06-17 05:06:29 UTC)。旧 plan/test 写的 `1781279189` 错了，**off by 393600s = 4.55 天**。错值不易发现：`resample_to_1hz` 用 `video_start_epoch + (0:N-1)` 算 `jpg_ts_epoch`，跟 `raw_ts` 匹配时 `argmin` 全 match 到第一个 raw sample，结果 400 行 `_1f.csv` 都跟第一个 GPS 一样，看不出 bug。

方案 Self-test 配方 (任何 .insv 都可)：

```
% 1. 用 exiftool + datetime 算出 video_start_epoch
% 2. 取 raw gps 的前几个 sample epoch
% 3. 算 delta: 期望 ~0-10s (GPS 锁星延迟)
% 4. 若 delta 异常大 (>100s)，八成是 video_start_epoch 错了
raw = posixtime(datetime('2026-06-17 05:06:36.6Z', ...
    'InputFormat', 'yyyy-MM-dd HH:mm:ss.S''Z'', 'TimeZone', 'UTC'));
delta = raw - 1781672789;
fprintf('delta: %.1f s\n', delta); % 期望 ~7.6
```

关键点 写测试或做 resample 时，不要复用旧 plan 的 1781279189。现算 (exiftool + datetime) 验证。

Why it matters: 若 `video_start_epoch` 错 4.5 天，所有 `jpg_ts` 都 off 393600s，GPS/IMU 1Hz 匹配静默失效。

08 IMU/GPS 时间对齐: mean_of_ends $\pm 200s$

IMU GPS 对齐

难点 GPS 是 UTC, IMU TimeCode 是 sensor-seconds-since-boot, trailer 头部**没显式 offset** (binary parser 能从 trailer 头拿到 boot timestamp, 但 exiftool 文本 dump 拿不到)。需要一种仅用 exiftool 的对齐方法。

方案 `insv_split.ps1` 用 **mean_of_ends** 方法估算 offset:

```
gps_mid = (gps_first_epoch + gps_last_epoch) / 2
imu_mid = (imu_first_timecode + imu_last_timecode) / 2
offset = gps_mid - imu_mid
gps_utc = imu_timecode + offset
```

假设: 两股流的中位数对应同一时刻 (录制在 IMU session 的中间附近)。Camera IMU 持续记录整个 session (开关机), GPS 只在录制期间 (含前后 pad) 有 fix。

误差界: $\pm \frac{1}{2} \times \min(\text{gps_span}, \text{imu_span}) / 2 \approx \pm \text{half of GPS span}$ (因为 GPS span 远小于 IMU span)。006 上 GPS span 6m40s, 所以误差 $\leq \pm 200s = \pm 3.3\text{min}$ 。

关键点 用户确认 $\pm 200s$ 精度暂时够用。改进方向: binary trailer parser 读 boot timestamp, 或用 video frame PTS + QuickTime CreateDate 锚 (两条都跟 mean_of_ends 不冲突, 可 override `utc_offset`)。

006 实测: offset = 1,778,314,012.285 s (~56.3 年 since 1970); IMU 完整 UTC 范围 02:20:24Z - 07:59:29Z, 跟实际相机 session 5.5h 吻合。

09 GPSTime混.X / 无小数 + duplicate ts 微秒偏移

GPS interp1 正则

难点

X5 GPSTime 数据两种格式混在一起:

1. 大多数行: 2026:06:17 05:06:36.6Z (1 位小数)

2. 偶发行: 2026:06:17 05:06:37Z (无小数)

gpx_filter.m 用 InputFormat='yyyy-MM-dd HH:mm:ss.S' 严格要求 1 位小数, 所以 416/4003 行 parse 失败 → NaN → interp1 报 "采样点必须为有限值"。

第二层 bug: 即便 pad 上 .0, 仍有 34 个 duplicate ts (exiftool 1-decimal 精度 print 相同, 实际相隔 ~200ms 但 lat 差 22cm)。不能 average (lat 是不同 sample 的物理测量, 车在动), 但

interp1 要 unique X → "采样点必须唯一"。

方案

两层 fix:

- **producer pad** (insv_split.ps1) : if (\$iso -notmatch '\.\d+Z\$') { \$iso = \$iso - replace 'Z\$', '.0Z' }

- **consumer 唯一化** (gpx_filter.m) : 用 unique+accumarray 给 dup 加 1e-6*i 秒偏移 (保留独立 lat/lon/ele)

关键点

数据 producer 是 X5 + exiftool, 格式不可控。Consumer 必须容错。微秒偏移不破坏数据但打破 unique 约束。

效果: 两层 fix 后 4003 GPS 全部能 parse + interp1, 34 个 duplicate ts 不丢物理信息。

10 exiftool IMU 输出 1Hz, 200Hz 需 binary parser

IMU binary parser

难点

exiftool 的 IMU 是 1Hz, 不是 200Hz。中位 dt = 1.0000s (X5 硬件是 ~200Hz, 但 exiftool 文本输出每 trailer group 只打 1 个样本)。13.35s clip 13.35×200 = 2,670 IMU 就够, 20k 没问题; 7min clip 会被 cap (~85,600 → 20,000)。

方案

要拿 200Hz 全量必须用 **binary trailer parser**:

- PyPI telemetry-parser (BenjaminHenriksson/insv-stitch 用的那个)

- 56-byte record 布局: [ts:u64 μs][gx,gy,gz:f64 rad/s][ax,ay,az:f64 m/s²]

- trailer 末尾 32-byte magic: 8db42d694ccc418790edff439fe026bf

完全 bypass exiftool, 最稳 (2-3 天工程量)。

关键点

-m 是 per-frame 抽取的开关 (401k samples 才是 per-frame, 20k 是 1Hz)。两者差 ~20×, 不要混。

何时触发: 短 clip (≤20s) 1Hz 够用; 长 clip (≥1min) 需要 binary parser, 否则 IMU 时间分辨率太低。

11 MediaSDK 公开 API 无 GPS，只能 exiftool

MediaSDK

API 调研

难点

想用 MediaSDK 抽 .insv GPS 是死路一条。 `grep -ri`

"GPS\|gpsData\|getMetadata\|getExtraInfo\|getGps\|getAcc\|getGyro" 在 3 个公开头里命中：

- `ins_stitcher.h`：只有 `EnableCuda`、`SetImageProcessingAccelType` 等 stitch 参数
- `ins_realtime_stitcher.h`：有 `HandleGyroData(const std::vector<GyroData>&)`，但这是**实时摄像头接 USB 推 gyro**，不是从 .insv 抽
- `ins_common.h`：`GyroData` struct (timestamp + ax/ay/az + gx/gy/gz)，仍是 live camera 数据结构

`VideoStitcher` 内部确实解析 .insv trailer (不解析没法 stitch)，但解析结果只用于内部 `FlowState` / `directionlock`，**不外暴**。`SetStabDataOutputPath` API 调成功但 0 字节。

方案

.insv GPS/IMU 抽取只能：

1. exiftool 读 .insv trailer (13.59 撞 perl bug)
2. exiftool 读 .lrv (低清代理，256MB，偶尔行)
3. Insta360 Studio GUI 手导出 → GPX (每批 .insv 点 1 次)
4. C++ 直接解析 .insv trailer binary (telemetry-parser 方案，工程量大)
5. 第三方 inSV 解析器 (frosted, exif-asf)

SDK 抽帧没问题 (走 `MediaSDKTest.exe`)，但抽出来的 jpg 不带 EXIF，GPS 得自己写。

关键点

任何想用 MediaSDK 抽 GPS 的尝试，先 grep 公开头确认有没有 API，没有就直接放弃走 exiftool。别花时间写 C++ 调 MediaSDK 内部 (私有 API 改版本就崩)。

决策：放弃 MediaSDK 路径，pipeline 已定型为 exiftool (抽 GPS/IMU) + MediaSDKTest (抽 jpg) 双工具组合。

12 stitch_to_frames 崩 → MediaSDKTest.exe (vulkan+CPU)

MediaSDK 抽帧 CUDA

难点 Insta360 MediaSDK 3.1.3 的官方 sticher (`stitch_to_frames.exe` 自带 wrapper) 在 NVIDIA 596.36 box 上崩 (GL crash `0xC0000139` / `0xC0000005`)。之前结论是“本机永远 fail, 需换机器或等新 driver”。

方案 同一份 MediaSDK 3.1.3 还带了一个**官方 sample binary** `MediaSDKTest.exe` , 默认走 vulkan + CPU, **不依赖 CUDA**, 0 崩。命令:

```
"C:\...\MediaSDKTest.exe" ^  
-inputs "X:\path\to\file.insv" ^  
-image_sequence_dir "X:\out\frames" ^  
-output_size 7680x3840 ^  
-enable_soft_decode ^  
-image_type jpg
```

可选: `-export_frame_index "0-30-60-..."` (1Hz timelapse 取子集) ; `-disable_cuda` (已默认) ; `-stitch_type template|optflow|dynamicstitch|aistitch` (默认 optflow, template 最快)。

DLL 依赖: 需要把整个 bin 目录加到 PATH (vulkan / cublas / cufft / npp / opencv / MNN / exiv2 / MediaSDK / CameraSDK)。

关键点 推翻旧结论 "sticher 在本机永远 fail"。任何 .insv 抽帧任务直接 `extract_jpgs(insv, out_dir, opts, 0)` 不再卡 env。

实测: 006 (13.35s @ 30fps 7680×3840) 抽全帧 400 张 157s; 抽 13 帧 (1Hz 子集) 14.4s; .lrv 5 帧 0.78s。8K 抽全帧慢 (~0.4s/帧 CPU), 抽 1Hz 子集用 `-export_frame_index` 提速。

13 insv_split.ps1 GPSTime 正则锚定 Z

PowerShell

正则

解析

难点 GPSTime 行 (2026:06:17 05:06:36.6Z) 用 `\[Insta360\]\s+GPSTime\s+:\s+(.+)Z\s*$` 这种 non-greedy 写法, `$Matches[1]` 在 2026:06:17 05:06:36.6Z 上只匹配到 2 (regex engine 取最短能让 `\s*$` 成立的串)。结果所有 Time 全是 1 字符, `Sort-Object` 排序完全乱。

方案 正确写法是锚定 Z :

```
\[Insta360\]\s+GPSTime\s+:\s+(.+)Z\s*$
$timeStr = $Matches[1] + 'Z' # reattach Z
```

用 greedy `(.+)Z\s*$` 锚定, 避免 non-greedy 短路。

关键点 解析行尾有固定 marker 的字段, 优先锚定 marker, 不要用 non-greedy `.+?` + 通配符结尾。

效果: 4003 行 GPSTime 全部正确解析为完整时间字符串 (如 2026:06:17 05:06:36.6Z)。

14 GPS 时间排序: 用秒 since epoch 而非 lex sort

PowerShell

排序

时间戳

难点 GPS 时间不能 lex sort。同一秒同时存在 05:13:16.9Z (1 位小数) 和 05:13:16Z (无小数) 两种格式, `. ASCII 0x2E < Z 0x5A`, lex 上 `05:13:16.9Z < 05:13:16Z`, 但时间上反过来。`Sort-Object` 按字符串排会乱。

方案 用 [double] SortKey (秒 since 1970-01-01 UTC) 排序, insv_split.ps1 里的 Get-GpsSortKey :

```
$epoch = [datetime]::SpecifyKind(
    [datetime]::ParseExact(
        $Matches[1]+'Z',
        'yyyy:MM:dd HH:mm:ss.fffZ',
        [Globalization.CultureInfo]::InvariantCulture),
    'UTC').ToUniversalTime()
$epoch.Subtract([datetime]'1970-01-01').TotalSeconds
```

对所有 GPS 行先 pad `.fff` (缺位补 0) 再算 epoch, 按 double 排序。

关键点 涉及时间排序的场景 (GPS/IMU/日志), 永远用 epoch 数值, 不用字符串 lex sort。

旧 parser bug (已被取代) : 4 个 elseif 分支各自 if (`-not $pts.Contains($lastTs)`) 创建新 entry, 但因 (1) Time lex sort 错乱, (2) `(\d+)` 抓秒只取整数丢小数 (58.50 → 58.00 → 36.71611 跟 36.71625 的区别), 输出 CSV 的 lat/lon 全卡在 36.7161111, 116.9375000。

15 PowerShell 5.1 不支持 PS6+ `?.` 运算符

PowerShell

兼容性

难点

`run_split_006.ps1` line 22 用了 PS6+ 的 null-conditional `?.`，默认 `powershell.exe 5.1` 直接 parser error。但 `MediaSDKTest.exe` 调用需要 PowerShell 5.1（Windows 自带），且部分工作站升级不到 PS7。

方案

改用 if/else 显式 null check：

```
# NOT OK (PS6+ only)
$obj?.Field

# OK (5.1 compatible)
if ($null -ne $obj) {
    $obj.Field
}
```

任何要跨 PS5.1/PS7 跑的脚本，禁用 `?.` / `??` / `?[]` 等 PS6+ 运算符。

关键点

解析层跨工具链脚本默认按 PS5.1 写。引入新语法前先 `$PSVersionTable.PSVersion` 检查。

触发条件：任何 `powershell.exe`（Windows 默认 5.1）调用的 `.ps1` 脚本。

C MATLAB 工程坑 (5 条)

16 MATLAB fopen 写 binary 0xFF 0xD8 被吃 (UTF-16 误判)

MATLAB binary IO .NET

难点 写 8 bytes `uint8([255 216 1 2 3 4 255 217])` (JPEG SOI/EOI) 到一个 .bin 文件, 用 MATLAB `fopen(file, 'wb')` 或 `'w','n'` 或 `'w','ieee-le','US-ASCII'`, **文件 on disk 都是 24+ bytes**, `0xFF 0xD8` 被替换成 `EF A3 B5 EF BF BD` (UTF-8 编码的 U+FFFD)。

根因: MATLAB 的 `fopen` 在 Windows 上把字节流当 UTF-16 处理, `0xFF 0xD8` 被识别为 UTF-16 high surrogate pair (无效上下文), 替换成 UTF-8 replacement character。

方案 完全 bypass `fopen`, 用 .NET `System.IO.FileStream` + `BinaryWriter` :

```
fs = System.IO.FileStream(path, System.IO.FileMode.Create, System.IO.FileAccess.Write);
bw = System.IO.BinaryWriter(fs);
bw.Write(uint8_array);
bw.Close();
```

同样读用 `BinaryReader.ReadBytes(N)` (注意 `ReadBytes` 返回 `System.Byte[]`, MATLAB 自动 cast 成 `uint8` 但有时要显式 `uint8(...)`)。

关键点 这两个坑叠加在 `pack_jpgs_bin` (写 .bin) + `extract_jpgs` (写 .jpg) 上。不解决等于把 **JPEG 全毁掉**。任何写 raw binary (jpg / png / protobuf / 自定义格式) 的 MATLAB 代码都走 .NET。

教训: 新写 MATLAB 写 binary file 的代码, 直接用 .NET, 别试 `fopen` 的 mode 组合。

17 MATLAB fread 'uint64' 只前进 4 字节

MATLAB binary IO typecast

难点 `fread(fid, 1, 'uint64')` 实际只读 4 字节, 像是被当 `uint32` 处理。读 1 `uint64` 后 `ftell(fid) = 4`, 不是 8。解析 binary header (如 .bin 的 offset table) 时悄无声息错位。

方案 不用 `fread` 的 `uint64` precision。改成读 2 个 `uint32` 然后 `typecast` :

```
lo = fread(fid, 1, 'uint32');
hi = fread(fid, 1, 'uint32');
val = typecast(uint32([lo hi]), 'uint64');
```

或者直接 `fread(fid, Inf, '*uint8')` 读全部字节, 自己 `typecast`。

关键点 任何 binary header 解析, 先用 `ftell` 验证前进字节数符合预期, 再继续。MATLAB `fread` 的 precision 字符串行为在不同版本差异大。

替代方案: 彻底用 .NET `BinaryReader` 读 `uint64` (避免 `fread` 坑), 统一 binary IO 栈。

18 .NET BinaryReader.ReadBytes 大文件 (>10MB) OOM

MATLAB

.NET interop

内存

难点

`System.IO.BinaryReader.ReadBytes(int32(fs.Length))` 把 18MB jpg slurp 成 `byte[]` 时抛 "外部组件发生异常。源: dotnetcli" (即 MATLAB-.NET bridge 崩, 不是真异常)。同样模式在 `File.ReadAllBytes(path)` 也有问题。

触发: 单次 `ReadBytes/ReadAllBytes` 返回的 `byte[]` > 10MB (经验值, 也可能 8MB 就崩, 看版本)。8K 7680×3840 jpg 典型 11–18MB, **全踩**。

Why: MATLAB R2026a .NET interop marshal 走 fixed buffer, 大 `byte[]` 超过 buffer 上限就 wrap 成 "External component exception"。不是真异常, **没法 try/catch 救**。

方案

不要 slurp 整个文件。用 **1MB buffer streaming**:

```
buf = uint8(zeros(1024*1024, 1)); % 1MB buffer
fs = System.IO.FileStream(fn, 'Open', 'Read');
remaining = fs.Length;
while remaining > 0
    chunk = int32(min(remaining, int64(numel(buf))));
    n = fs.Read(buf, 0, chunk);
    if n <= 0, break; end
    out.Write(buf(1:int32(n))); % buf slice + write
    remaining = remaining - int64(n);
end
fs.Close();
```

关键点

任何从 pipeline 处理 >10MB 文件的 .NET binary read 都要 streaming。适用 `pack_jpgs_bin` (写 bin)、`extract_pano_at_gps` (读 jpg) 等。永远不要 `ReadAllBytes(path)` 或 `ReadBytes(fs.Length)` 大文件。

坑区分: 跟 `[[matlab-fopen-windows-binary-pitfalls]]` 是不同坑 (fopen 是 text-mode mangling, .NET 是 bridge buffer overflow), 都得绕。

19 uigetfile MultiSelect 只接受 'on' / 'off' 字符串

MATLAB GUI uigetfile

难点 调用 `uigetfile(..., 'MultiSelect', mat2str(opts.MultiSelect))`, 当 `opts.MultiSelect=false` 时报:

错误使用 `uigetputfile_helper/parsePropertyValuePairs` (第 209 行)
'MultiSelect' 的有效值为 'on' 和 'off'。

`mat2str(false)` → 'false', 但 `uigetfile` 只认 'on' / 'off'。R2026a 没改这 API。

方案 写 helper `ternary_onoff.m`:

```
function s = ternary_onoff(b)
    if b, s = 'on'; else, s = 'off'; end
end
```

调用: `'MultiSelect', ternary_onoff(opts.MultiSelect)`。

替代方案对比:

- `char(string(b))` — R2026a `string(false)="false"`, 同样拒
- `num2str(b)` — `num2str(true)='1'`, 拒
- `if/else` inline — OK 但丑

关键点 任何把 logical 当字符串拼到 `uigetfile` / `uiputfile` / `uigetdir` 的 `MultiSelect` / `DefaultName` 等 on/off 字段的地方, 默认用 `ternary_onoff`。

验证: `ternary_onoff(false)` → 'off' ✓; `uigetfile` 不再报 valid values 错。

20 GUI 集成必改 prompt_idempotent.m 静默 questdlg

MATLAB

GUI 集成

questdlg

难点 第二次跑同一 .insv → `insv_to_project` 默认会弹 `questdlg [Skip / Overwrite / Cancel]` , 在 batch / 嵌入式 GUI 会卡死或被吞, 导致集成按钮"按下去没反应".

方案 改 `scripts/prompt_idempotent.m` 第 11 行 `choice = questdlg(...)` → `choice = 'overwrite'` :

```
% 强制覆盖, 不弹对话框
choice = 'overwrite';

% 或: 静默判断
choice = 'overwrite'; % 默认
% choice = 'skip';    % 永远跳过, GUI 启动时若想全量重跑再改
```

调用方拿到的 `stats.skipped` 会始终 false, 但二次跑会覆盖旧产物 (5.8 min) 。

关键点 GUI 集成必改 `prompt_idempotent.m`, 否则 `questdlg` 阻塞。完整集成模式 (6 步) :

1. 调 `insv_to_project(insv_path, ...)` (不要用 `insv_to_project_gui`, 会抢 `uigetfile`)
2. 必传 `ProjectRoot` 让 GUI 控制输出位置
3. 设 `Verbose=false` 避免内建 `fprintf` 污染
4. 用 `stats.skipped` 区分"幂等命中"vs"新跑完"
5. 锁按钮 (`Enable off→on`) 防重入
6. 改 `prompt_idempotent.m` (同上)

完整样板: `C:\i361\docs\INTEGRATION.md §2`。异步版用 `parfeval(@insv_to_project, 1, insv_path, ...) + afterEach`。

21 V30 uifigure 标题栏高度让位问题 (R2026a)

MATLAB

App Designer

uifigure

难点

V30.4 → V30.7 一天内迭代 4 个版本解决 uifigure 标题栏让位:

- **V30.4:** `UIH=1020`, `E=70`, 新增 `TitleBarH=40` 偏移所有区域避开标题栏
- **V30.5:** 修 figure Position 越界 `[1 1 1920 1060]` → `[1 1 1920 1020]` + startup diagnostic
- **V30.6:** R2026a uifigure `InnerPosition=Position` **无 figure title bar 让位**, `TitleBarH` 40→15
- **V30.7:** `TitleBarH` 15→0 (完全贴顶)

调优历史 40→15→0 的教训: uifigure 默认 `TitleBarH=0`, 前面的偏移全是从错误前提推导出来的。

方案

V30.8 敲定 **maximized 模式** (`Resize='on'` + `WindowState='maximized'`), **完全规避**标题栏/任务栏/DWM/多屏偏移的所有复杂计算:

```
fig = uifigure('WindowState', 'maximized', 'Resize', 'on');
% ScreenSize = [1 1 1920 1080]
% UIW=1920, UIH=1020, TitleBarH=0 (maximized 模式下都是 0)
```

后续所有 uifigure 项目通用方案。

关键点

uifigure 没有 `AutoResize` 属性 (与 legacy figure 不同)。 `InnerPosition` 在 R2026a 跟 `Position` 一致, 不预留 figure title bar。

V30.14 实测: [V30] `ScreenSize = [1 1 1920 1080]`, `UIW=1920`, `UIH=1020`, `TitleBarH=0`, 5 区域全部铺满零留白。

22 V30 WindowState='maximized' + 5 区域基线 + loadProject 统一入口

MATLAB App Designer 架构

难点

V30 一天内迭代 13 版 (V30.1 → V30.13) 敲定 UI 基线, V30.14 实现真实数据加载。挑战:

1. **UI 不变量**: 5 区域 (A 图像/B 地图/C 帧跳转/D 工程/E 信息栏) + maximized 模式 + panel 去 Title/Border (不显示区域名)
2. **回调签名坑**: 要访问 `obj` 属性时必须 `(obj, ~)`, 否则只能取 `~`
3. **集成 pipeline**: GUI 构造函数里 `addpath('C:\i361\scripts', '-begin')` (`-begin` 优先避免 v30 自带脚本冲突); `onNewProj` 调 `insv_to_project_gui()` → 11 步 pipeline → stats → 调 `loadProject`
4. **V30.14 loadProject 统一入口**: `onNewProj/onOpenProj` 都调 `loadProject`; 8 业务状态 + `refreshCurrentFrame`; 3 个新 helper 脚本 (`read_jpgx_bin` / `plot_gpx` / `refresh_state`)

方案

UI 基线 (不要再改) :

区域	位置	尺寸	内容
A	(1, 1)	1600×900	imshow 当前帧
B	(1620, 1)	300×600	plot GPS 轨迹 + 红点
C	(1620, 610)	300×140	2×2: 倍率/自动/上一帧/下一帧
D	(1620, 760)	300×140	2×3: 视图类 3 + 工程类 3
E	(1, 910)	1920×70	6 段: 帧号/时间/经纬度/输入/跳转

loadProject 统一入口 (V30.14) :

```
function loadProject(obj, prj_path)
    % 1. parse .prj JSON → stats
    % 2. set 8 业务状态 (curFrameIdx, gpxData, imuData, ...)
    % 3. 调 refreshCurrentFrame 刷 A/B/E
    % 4. 启用 C/D 区按钮
end
```

用户偏好 (拍板) :

- 不显示"区域名" (V30.9) : "调整各个区域, 并不需要单独表明这个什么区域"
- 完全贴顶零留白 (V30.7)
- 回调签名 `(obj, ~)` (要读 `obj` 属性时)

关键点

改 UI 前先看 `v30-ui-finalized` 标注的"不要再改的事"。改 pipeline 前先看 `INTEGRATION.md` 保证不影响集成样板。跨窗口知识共建/隔离 (`README.md` 协议) 。

V30.14 状态: D 区 `BtnNewProj` / `BtnOpenProj` / A 区 `imshow` / B 区 `plot` / E 区 4 信息 label 全部真实运行; C 区 4 元素 + D 区 3 元素 + E 区 `EditFieldJump` 留 V30.15+ 接真实功能。

23 离线地图坐标与 GPS 坐标系不对应，需转换坐标系

坐标系

wgs84

gcj02

gps

高德偏移

现象

同一段 GPS 轨迹，在微图 (WeMapGIS) / 高德 / 腾讯底图上**明显偏移** 500~1000m，在 OSM/Google 上才“看着对”。V30.15~V30.20 期间反复尝试，仍按 WGS84 直接画点 → 国内底图永远对不上；用户报告“在铁路上跑，轨迹却偏到马路另一边”。

根因

中国境内的电子地图 (高德、腾讯、百度、微图等) 法定使用 **GCJ02 (火星坐标)**，对真实 WGS84 经纬度施加伪随机加密偏移 (振幅 0.001~0.01°)；OSM/Google/ArcGIS 才是真 WGS84。GPS 设备输出的原始经纬度是 WGS84，**直接画在 GCJ02 底图上必错位**。

尝试

1. V30.16 直接 `plot(lon, lat)` → 错位 800m。
2. V30.18 怀疑是 mbtiles 切片坐标问题 → 改 YAxis.TMS → 错位方向变但仍在 600m。
3. V30.20 写 `wgs84_to_gcj02.m` (标准算法 $a=6378245$, $ee=0.00669342162296594323$) → 转换后轨迹**精确落在底图铁路上**。

方案

`wgs84_to_gcj02.m`：先判断是否在中国境内 (粗略经纬度矩形 73.66~135.05, 3.86~53.55)，在境内才转换；境外 GPS 保持 WGS84 不动。画点前先过此函数。再加 `test_v30_36_wgs84_vs_gcj02.m` 用京津、京沪段已知坐标做端到端回归。

踩坑

GCJ02 算法是公开常数但有几种近似实现，浮点精度差 ~1m；统一用 ICSM 公式 (不用百度 / 搜狗变体)，并对转换前后做对称 sanity check (往返误差应 $< 1e-7^\circ$)。

结论：任何 X5 项目代码**画轨迹前必须先过 WGS84→GCJ02 转换函数** (境内才转，境外不转)，否则用户在国内任何底图上看到的都是错位轨迹。这是用户反馈“一开始的方法没对应上”的根因。

24 Mapsui 五个实测坑：从 WPF 控件切到手动 RenderToBitmapStream

mapsui brutile wpf csharp tms

现象 V30.21~V30.30 期间尝试用 Mapsui 5.x 渲染底图+轨迹，五个独立问题反复造成"底图空白"、"整图白屏"、"标记消失"、"加载崩"。每个问题都得反编译 IL 才能定位。

- 五个坑**
1. **WPF MapControl 的 TileLayer 异步 fetch 不触发** → 底图永远空白。反编译 `MapControl.Refresh` IL 证实它只刷 graphics、不触发 tile fetch。修法：改用 `MapRenderer.RenderToBitmapStream(map, 1f, RenderFormat.Png, maxIterations:6)` 手动渲染到 WPF Image。
 2. **Polygon 渲染导致整图空白**（仅 Outline 无 Fill 时尤甚）。KML/边界改用 `LineString` 画闭合线，效果一样且能渲染。
 3. **SymbolStyle.SymbolScale < 1 渲染退化**（标记消失/变白）。标记大小一律 `SymbolScale >= 1`；`DefaultWidth/DefaultHeight` 是 Mapsui 静态属性（全局默认 32px），不能在对象初始化器里改。
 4. **版本必须匹配**：Mapsui 5.x 依赖 BruTile 6.0.0；`BruTile.MbTiles` 必须同为 6.0.0，否则 `TypeLoadException: get_Attribution`。查传递依赖：`dotnet list package --include-transitive`。
 5. **国内微图 mbtiles 多为 TMS 方案**（tile_row 0 在南）：
`GlobalSphericalMercator(YAxis.TMS, 12, 18, "weMap")` 的 `YAxis` 参数必须选 TMS（OSM 是 0 在北）。

- 诊断方法论**
- 反编译 `MethodBody.GetILAsByteArray()` 逐字节看 `call` 目标
 - headless 对照组：`RenderToBitmapStream` 绕过 UI，快速区分"数据问题 vs 控件问题"
 - 逐 feature 二分：单个叠加要素都成功、组合失败 → 锁死具体 feature
 - 版本冲突：`dotnet list package --include-transitive`

方案 V30.30+ 统一改用 `MapRenderer.RenderToBitmapStream` + `LineString` + `SymbolScale>=1` + Mapsui 5.x + BruTile 6.0.0 + `YAxis.TMS`。这套组合终于让底图+轨迹正确套合。

结论：Mapsui 是个功能强大但坑很多的库；遇到任何"底图空白"先查这五项。修复完用户验收："轨迹终于和铁路对上"了。

25 V30.24+ 八连 bug：从 char() 解析到 obj.mapAxes.XLim 点索引

v30

matlab

classdef

patch

axes

背景

V30.23→V30.24+ 的“打开离线地图”按钮 patch 引入 8 个新 bug（不是 1 个），每个 bug 都需要单独修。完整诊断会话 2026-08-25 凌晨进行（`v30-bugfix-round2-session-20260825_041620.md`）。

八个 bug

1. **Bug 1:** `lblMapStatus.Text` 行 305/306 的 `char()` 把 `figure` 函数名当字符串解析（编码问题）。改 hex 编码解决。
2. **Bug 2:** `createRegionC/D/E` 三段代码被原 patch 误删（patch diff 没考虑新增方法）。从 V30.23 备份恢复。
3. **Bug 3:** `refreshCurrentFrame` 悬空引用 `bmapLineHandle` 等（变量已被 patch 改名）。改为调用 `obj.refreshOfflineMap()`。
4. **Bug 4:** 离线地图白色背景 + 轨迹消失（高频 `imshow` 全量重绘）。改用 `set(XData/YData)` 部分更新 + lock `XLim/YLim`。
5. **Bug 5:** 底图上下颠倒（`imshow` 默认 `YDir='reverse'`）。行 526 `YData` 改为 `[36.7383 36.6504]`（北→南反序传入）。
6. **Bug 6:** 离线地图太小无法缩放。默认 `XLim/YLim = 轨迹 bounds + 5% pad`，加 `zoom/pan` 回调。
7. **Bug 7:** `onSpeedChange` 方法缺失（timer 重建时回调引用未定义）。新增方法（`destroy→create timer pattern`）。
8. **Bug 8:** `obj.mapAxes.XLim = ...` 点索引报错。根因：行 56 `mapAxes double` 类型强转导致 axes 对象被当成数值句柄 → 点索引失败。方案 A（推荐）：删 `double` 类型标注，与 `mapImageHandle` 一致。

方案

Bug 1-7 已全部修完并部署；Bug 8 走方案 A（删 1 行类型）后即生效，需 MATLAB 重启（`clear classes`）让 classdef 重新加载。验证：自动播放时 GPS 红点逐帧跟随 + 改速度下拉不报 `onSpeedChange` + 检查 `matlab_v30.log` 无新错误。

教训

- patch 引入的 bug 经常不是 1 个，要按“行为→报错→行号→classdef→git diff”五步逐项排查。
- MATLAB classdef property 类型一旦声明强类型，赋值时 MATLAB 默默转换，运行时才报点索引错误——**不声明类型最安全**。
- `imshow` + `YDir='normal'` 必须 `YData` 反序传入（北纬更大）；不反序则底图永远颠倒。

结论： V30.24+ 离线地图按钮从“点开报错”演进到“轨迹与铁路精确对齐”，全程跨 8 个 bug + 5 轮 patch。这是用户口中“经过多种尝试，对应上了”的核心故事。

26 手写 Web Mercator 投影：纬度方向偏差与 sin/cos 近似坑

mercator

projection

tile-math

anti-pattern

现象 V30.16~V30.22 期间，自己手写了 `TileToLat` / `LonLatToTile` / `MercY` / `latRange = mercYRange * cosLat` 一套“够用就行的”投影。结果是经度方向精确对齐，**纬度方向永远偏**，偏移在低纬度 ~50m / 高纬度 ~500m，正好和 GCJ02 偏移叠加 → 完全不可调试。

根因 Web Mercator 经度线性、纬度非线性： $y = R \cdot \ln(\tan(\pi/4 + \phi/2))$ 。手写时三段独立近似：①瓦片编号换算 ②视口范围计算 ③叠加层投影映射。任何一处不一致就错位，且**集中在纬度方向**。同时 `cos(lat)` 把矩形投影回地理范围时也是近似（地球是椭球而非球），低纬度 ok 高纬度差几百米。

失败案例

- `verify_tile_math.py`：用 `python -m pyproj` 对照发现手写投影纬度差 0.003° （约 330m @ 纬度 38° ）。
- `compare_basemaps.py`：同一 GPS 点在自写投影 vs OSM 切片上差 ~5 像素（zoom 18 下 ~5m），在 zoom 14 下放大到 ~80m。
- `diag_y_flip_compare.m`：证明 YData 反序 vs YDir='normal' 必须二选一，混用必颠倒。

方案

1. **短痛**：直接用成熟库（Mapsui 5.x / maptiler / leaflet / mapbox），库不让手写投影所以不会错。
2. **若必须手写**：用 EPSG.io / PROJ 公式 + 椭球参数 WGS84 ($a=6378137$, $f=1/298.257223563$)，不用球面近似。
3. **回归测试**：`verify_tile_math.py` 用已知地理点（天安门 116.3974, 39.9092）做端到端对照，**不允许 > 1m 误差**。

结论：地图投影是典型的“看着简单实际极难”的领域，**永远用库**。手写任何 Web Mercator 都得跑三组对照（已知点 / 库实现 / 真实切片），成本比直接用库高 5 倍。

27 30+ 诊断脚本的五轮迭代：从猜到证据链

diagnostic

iteration

scripts

evidence

背景

C:\last\last\scripts\ 积累了 30+ 个地图/轨迹/底图相关的诊断脚本，每个对应一轮“假设→证据→新假设”循环。这是用户口中“多种尝试”的具体呈现。

五轮迭代

1. **第 1 轮 (V30.16 前后) — 切投影猜 Y 方向：** `diag_y_flip.m` / `diag_y_flip_compare.m` → 发现 YData 反序必须显式做。
2. **第 2 轮 (V30.18 前后) — 切 YAxis 猜切片约定：** `diag_tms_vs_osm.m` → 微图是 TMS (0 在南)，与 OSM 相反。
3. **第 3 轮 (V30.20 前后) — WGS84→GCJ02 偏移假设：** `wgs84_to_gcj02.m` + `test_v30_36_wgs84_vs_gcj02.m` + `compare_basemaps.py` → 验证国内底图偏 500~1000m 的真因是 GCJ02 加密。
4. **第 4 轮 (V30.24~V30.30 前后) — Mapsui 黑盒 bug：** `render_basemap.m` (headless 对照组) + `offlineMap.m` (绕过 WPF 控件) + `probe_mbtiles.m` (检查 SQLite 切片数据) → 定位到 WPF MapControl 不 fetch tile。
5. **第 5 轮 (V30.30~V30.36+) — 误差收口：** `zoom_match.py` (不同 zoom 误差放大) + `verify_tile_math.py` (库实现对照) + `diag_offset_log.txt` (迭代日志) → 误差收敛到 < 1m，轨迹终于和铁路对齐。

方法论沉淀

- **诊断脚本要带时间戳** (`diag_y_flip_compare_20260824.m`)，便于回溯思路变化。
- **每轮假设单独一个脚本** (不混在 pipeline 里)，`scripts/` 目录本身就是一份“我们试过什么”的清单。
- **对照三组：** 自写实现 vs 库实现 vs 真实切片数据，三者一致才认。
- **headless 先行：** UI 控件往往掩盖真因，先用 `RenderToBitmapStream` 这类无 UI 路径验证。
- **offset_log 不删：** 每次“修好了”留 `offset_log_20260918.txt`，后来 V30.36 重现 bug 时直接对照历史 log。

最终态

V30.36+ 用户验收：“在铁路上跑，轨迹就是这条线”。

结论：“对应上了”不是一句结论，是 30+ 脚本、5 轮假设、跨 14 个版本迭代出来的可执行证据链。

28 自带 SDK 不支持多线程，自实现 4 线程 wrapper (1~16 线程对比实测)

parallelstitcher

cpp

sdk

multithreading

bench

v60

现象

Insta360 MediaSDK 3.1.3 自带的官方 stitcher (`MediaSDKTest.exe`) 是**单进程串行**的：对一个 .insv 抽 N 帧，内部一帧一帧串行调用 MediaSDK.dll，400 帧需要 ~200 秒。同一份 SDK 资源已经在 GPU 上，但单线程限制让 GPU 大量空闲。

根因

MediaSDK 是 C++ 动态库，**没有公开的多帧并发 API**：进程内多线程调用同一 MediaSDK handle 互斥（内部 mutex 串行化），多进程调用同一 GPU 上下文会触发 OpenCL/Vulkan 资源争抢。所以“在 V30 里开 8 个线程调 SDK”看似并发，实际内部还是排队。

方案

写一个 **C++ 多进程 wrapper** `ParallelStitcher.exe`：

- 每个 worker 是独立 OS 进程，**各自加载 MediaSDK.dll、独立 GPU context**，绕过 SDK 内部 mutex。
- 主进程按帧区间 (`FramesPerChunk=100`) 切分任务，pipe stdout 收集各 worker 完成帧。
- **PATH 注入**：子进程 PATH 必须包含 SDK 目录（含 MNN.dll / OpenCL.dll），否则 `LoadLibrary:126` 失败。
- **stale-jpg 清理**：启动前 `rm chunk_*/*.jpg`，否则 init race (worker 0s 完成，MediaSDK 还在 async-init)。
- orchestrator (`V30.Stitcher.Stitcher`) 做 batch retry (最多 3 次) + manifest resume + GPU watchdog。

线程数对比

(1~16 全跑)

`V30.Stitcher.Tools.Bench` 20-run 基准测试，399 帧 / 7680×3840 / NVIDIA 上：

- **1 线程**：~200s (baseline, 等同旧 MediaSDKTest)
- **2 线程**：~140s
- **4 线程**：~72s (**2.78× speedup, 20/20 success**) ★ **最优**
- **6 线程**：~95s (开始 GPU 争抢，下降)
- **8 线程**：~680s (GPU contention 700× 退化, `StitchOptions.Validate()` 拒绝)
- **12/16 线程**：同样被 Validate 拒绝 (>4 全禁)，bench 不跑

关键约束

- `Threads ∈ [1, 4]`：8 线程触发 GPU 争抢，单帧耗时从 ~180ms 退化到 ~125s (700× 慢)。
- JPG mode only: H265 mode 在 dtor 里阻塞 200s 刷编码器，未暴露。
- `Process.ExitCode == 0` 不可信：MediaSDK 会 `ExitProcess(0)` 假成功。判定标准：stdout 完整 + 所有 chunk 帧完成。

验证

`dotnet run --project src/V30.Stitcher.Tools.Bench -c Release -- --runs 20 --threads 4`

实测：mean=72.0s ± 0.27s, CV 0.38%, 20/20 successes。79 个 unit test 通过，1 个 E2E skip。

结论：SDK 没自带多线程 → 自己写 C++ wrapper 把“单进程串行”拆成“多进程并行 + 独立 GPU context” → 实测 **4 线程是最优甜点** (再高 GPU 争抢，再低 CPU 空闲)。这条经验直接决定了 V30-60 release pkg 的 Stitcher 形态。